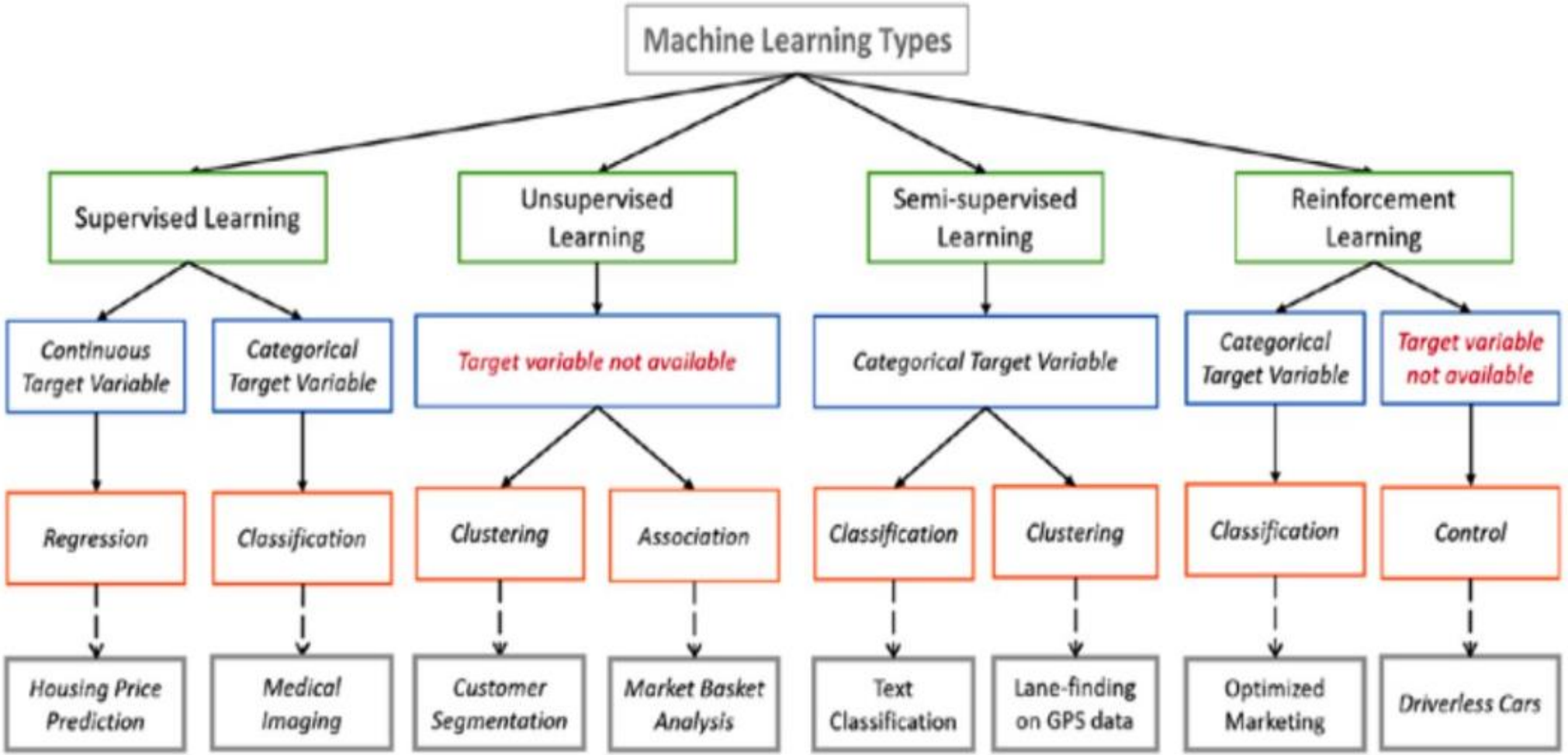
Regressão linear simples e múltipla com python

Fatec 2026

Tipos de aprendizado de máquina



Variáveis

Variável Independente

(ou explicativa, preditora)

- É a variável que controlamos ou observamos para explicar ou prever outra
- Representa a causa ou o fator que influencia a outra variável
- Fica do lado direito da fórmula do modelo

Exemplo: `horas_estudo`

Quanto mais horas a pessoa estuda, maior tende a ser sua nota.



influencia

Variável Dependente

(ou resposta, regressanda)

- É a variável que queremos prever ou explicar
- Representa o efeito ou o resultado influenciado pela variável independente
- Fica do lado esquerdo da fórmula do modelo

Exemplo: `nota_prova`

A nota depende das horas de estudo.

Regressão linear

- A Regressão Linear permite gerar um modelo matemático através de uma reta que explique a relação linear entre variáveis
- No caso mais simples, teremos a relação entre uma variável explicativa X (independente) e uma variável resposta Y (dependente)
- Outro fator bastante positivo da regressão linear, e que faz dela um algoritmo
- muito utilizado tanto na estatística “tradicional” quanto em machine learning, é que o algoritmo nos traz informações inferenciais, ou seja, podemos extrapolar conclusões para a população a partir da amostra e também podemos utilizá-lo para modelagem preditiva

smf.ols() – Modelo linear (OLS)

- A função smf.ols() da biblioteca statsmodels ajusta modelos lineares, como **regressão linear simples** ou **múltipla**, usando o método dos **mínimos quadrados ordinários**
- Sintaxe:
  - **`smf.ols(formula, data, subset, missing, ...).fit()`**

Argumento	Descrição
`formula`	A equação do modelo. Ex: 'y ~ x1 + x2' significa “y em função de x1 e x2”
`data`	DataFrame (pandas) onde as variáveis estão definidas
`subset`	Subconjunto das linhas a ser usado
`missing`	Como tratar valores ausentes (NaN): 'none', 'drop' ou 'raise'
`C(x)`	Dentro da fórmula, marca x como variável categórica
`.fit()`	Ajusta o modelo (estima os coeficientes) e devolve os resultados

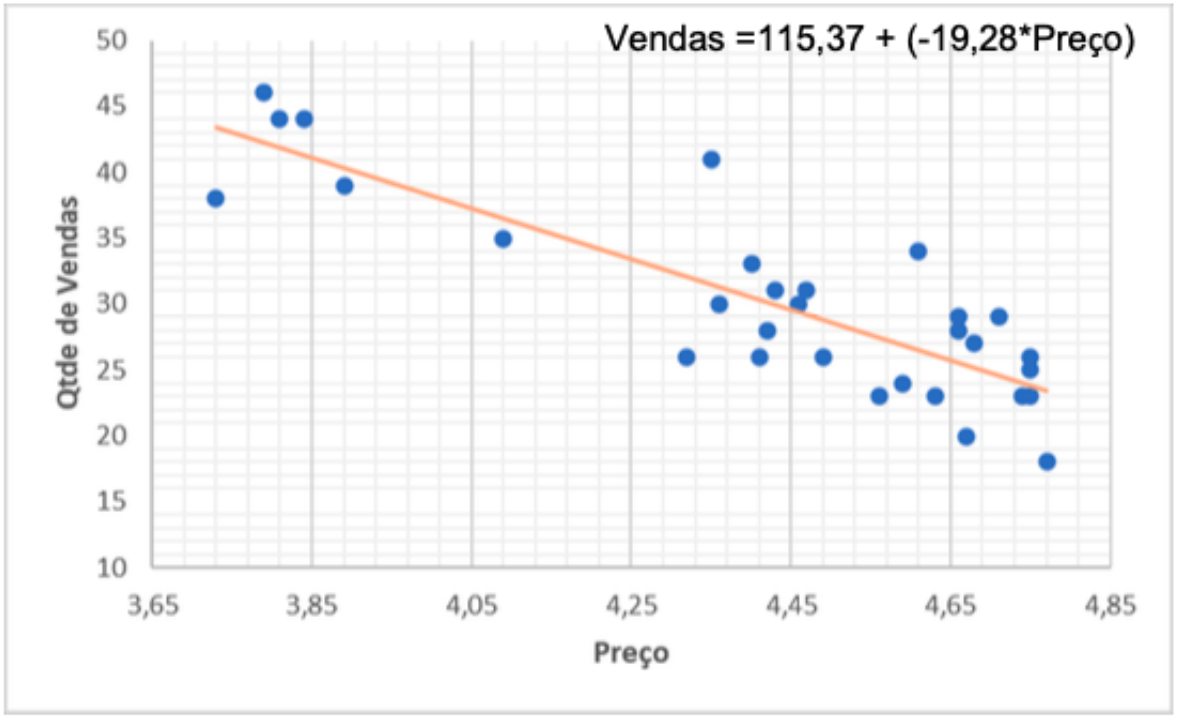
Retorno:

Um objeto de resultados (RegressionResults) que contém:

- Coeficientes (intercepto e inclinações): .params
- Resíduos: .resid
- Valores ajustados (fitted): .fittedvalues
- Diagnóstico: .summary(), .rsquared, .pvalues

Regressão linear Simples

- A regressão linear simples é quando temos apenas uma variável preditora e uma variável resposta



$$Vendas\ do\ Café = 115{,}37 + (-19{,}28 * Preço\ do\ Café)$$

Onde 115,37 é o intercepto B_0 e -19,28 é o coeficiente angular B_1. Ou seja, a cada real que aumenta no preço, as vendas caem, em média, em 19,28 unidades.

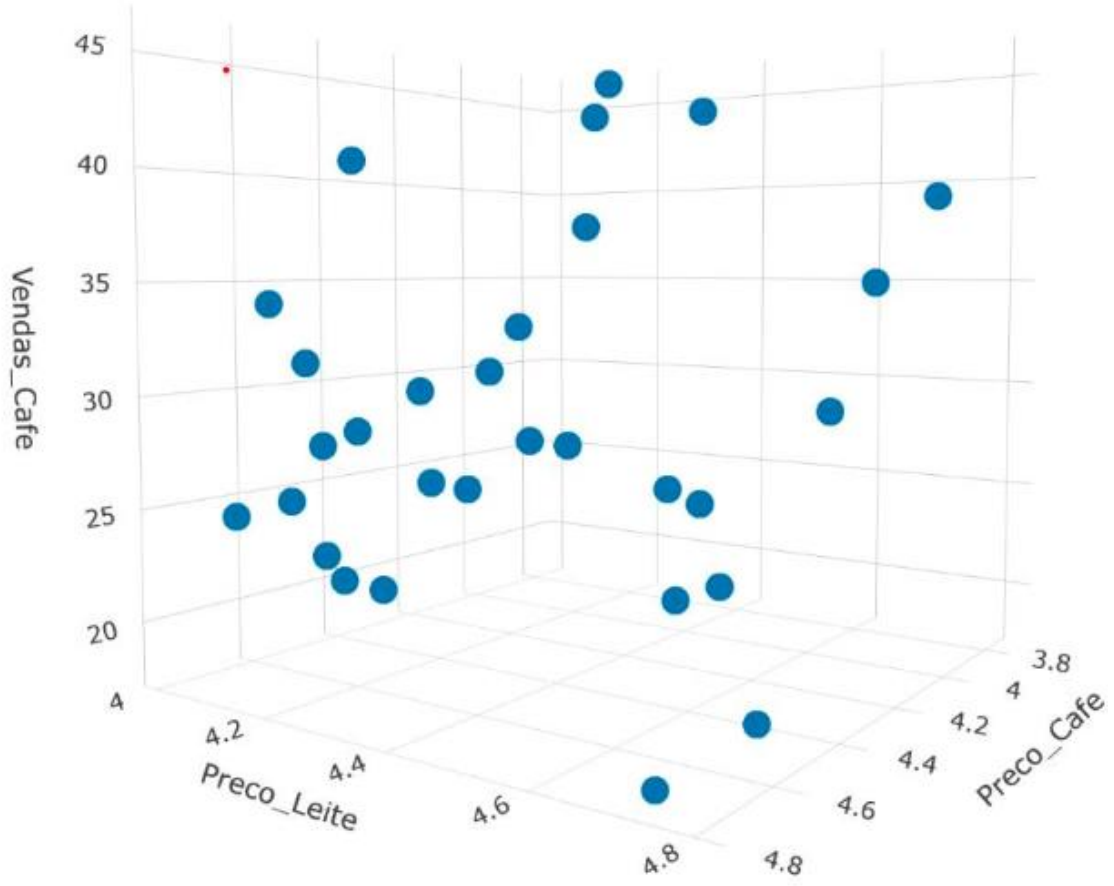
Regressão linear múltipla

- Quando se inclui mais uma variável independente
- Por exemplo, suspeita-se que na nossa empresa, o leite seja um produto complementar ao café, ou seja, o cliente que compra café tende a comprar leite
  - Se essa hipótese for verdadeira, ao realizar mudanças no preço do leite as vendas do café também poderão ser impactadas
  - Ao incluir mais uma variável no modelo (preço do leite) estamos trabalhando com três variáveis:

    duas variáveis independentes (preço do café, preço do leite) e

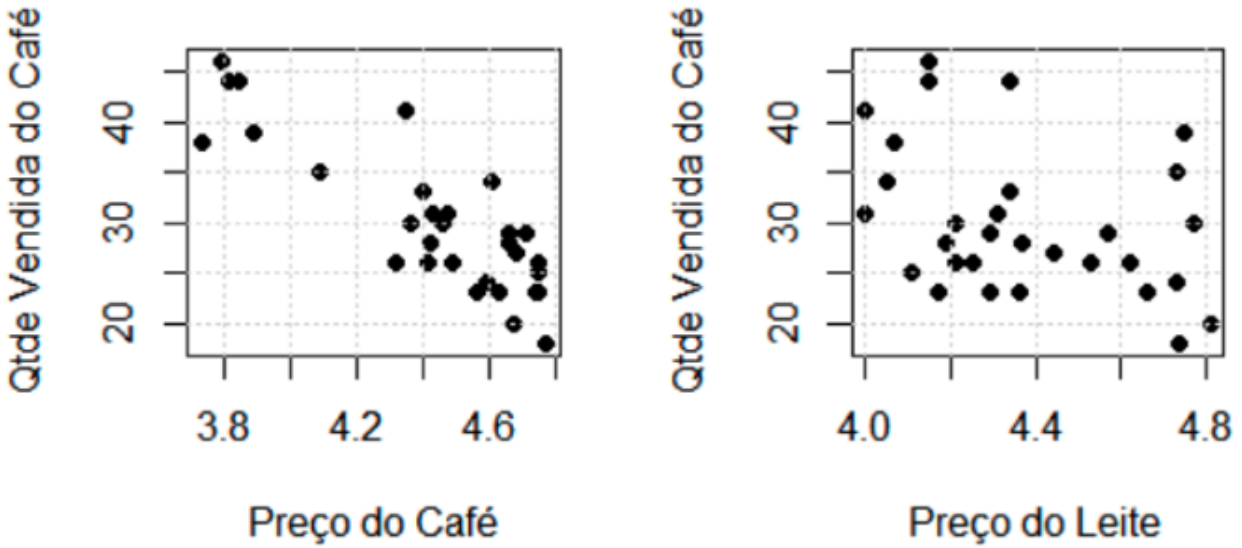
    uma variável dependente (quantidade vendida do café)

Regressão linear múltipla



É possível ver que quando o preço do leite aumenta as vendas do café caem,
assim como quando o preço do café aumenta as vendas do café também caem.

Regressão linear múltipla



A equação de regressão múltipla ficou:

$$Vendas\ do\ Café = 151{,}31 + (-18{,}72 * Preço\ do\ Café) + (-8{,}78 * Preço\ do\ Leite)$$

Projeto Cafeteria

```
# Importação das bibliotecas necessárias
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import statsmodels.formula.api as smf

# Configurações visuais padrão
plt.rcParams['figure.figsize'] = (8, 5)
sns.set_theme(style='whitegrid')
```

No notebook: se o arquivo dados_cafeteria.csv ainda não estiver carregado no Google Colab, a célula gera automaticamente uma **base simulada equivalente**, para que todo o código possa ser executado sem erros.

```
# Carregar os dados a partir de um arquivo CSV
dados = pd.read_csv('dados_cafeteria.csv')
# Ver os primeiros registros
dados.head()
```

Projeto Cafeteria : Estatística descritiva

```
### Conhecendo os dados ###
# Como estão distribuídos os preços e as vendas?

# Resumo estatístico (média, desvio padrão, quartis, mínimo e máximo)
print(dados.describe())

# Visualizando a distribuição das variáveis
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
sns.histplot(dados['Vendas_Cafe'], kde=True, color='skyblue', ax=axes[0])
axes[0].set_title('Distribuição das Vendas de Café')

sns.histplot(dados['Preco_Cafe'], kde=True, color='salmon', ax=axes[1])
axes[1].set_title('Distribuição dos Preços do Café')

sns.histplot(dados['Preco_Leite'], kde=True, color='lightgreen', ax=axes[2])
axes[2].set_title('Distribuição dos Preços do Leite')

plt.tight_layout()
plt.show()
```

Projeto Cafeteria

```
### Preço influencia vendas? ###
# Observar visualmente a relação entre preço e quantidade vendida.

plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=dados,
    x='Preco_Cafe',
    y='Vendas_Cafe',
    color='blue',
    s=80
)
plt.title('Preço vs Vendas de Café')
plt.xlabel('Preço do Café (R$)')
plt.ylabel('Vendas de Café (unidades)')
plt.show()
```

No seaborn, as **linhas de grade do gráfico** vêm do tema definido em **sns.set_theme(style='whitegrid')**. No matplotlib puro, use plt.grid(True).

Projeto Cafeteria

```
### E as promoções, fazem diferença? ###
# Pontos em vermelho quando há promoção.

plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=dados,
    x='Preco_Cafe',
    y='Vendas_Cafe',
    hue='Promocao',
    palette={'Sim': 'red', 'Nao': 'black'},
    s=80
)
plt.title('Impacto do Preço e da Promoção nas Vendas de Café')
plt.xlabel('Preço do Café (R$)')
plt.ylabel('Vendas de Café (unidades)')
plt.show()
```

Projeto Cafeteria

```
### Acima ou abaixo da média? ###
# Quantas vezes as vendas ficaram acima ou abaixo da média histórica?

media_vendas = dados['Vendas_Cafe'].mean()
dados['Status_Media'] = np.where(dados['Vendas_Cafe'] > media_vendas,
                                 'Acima da média', 'Abaixo da média')

print(f'Média histórica de vendas: {media_vendas:.2f}')
print(dados['Status_Media'].value_counts())

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.countplot(data=dados, x='Status_Media', palette=['gray', 'orange'], ax=axes[0])
axes[0].set_title('Vendas em Relação à Média')
```

O método value_counts() do pandas cria uma **tabela de frequências absolutas** — ou seja, ele **conta quantas vezes cada valor único aparece em uma variável**

Projeto Cafeteria

```
### Diferença entre dias com e sem promoção ###
# (continuação da mesma figura: segundo painel)
sns.boxplot(data=dados, x='Promocao', y='Vendas_Cafe',
            palette=['lightgray', 'salmon'], ax=axes[1])
axes[1].set_title('Vendas com Promoção vs Sem Promoção')

plt.tight_layout()
plt.show()
```

Projeto Cafeteria: Regressão Linear

Argumentos mais usados em sns.lmplot():

Argumento	Significado
`data`	DataFrame com os dados
`x, y`	Colunas dos eixos x e y
`hue`	Coluna que define os grupos (cores)
`palette`	Cor de cada grupo
`fit_reg`	True/False: desenha ou não a reta de regressão
`height, aspect`	Altura e proporção da figura

```
### Regressão linear simples ###
# Vamos modelar a relação entre preço do café e vendas.
modelo_simples_cafe = smf.ols('Vendas_Cafe ~ Preco_Cafe', data=dados).fit()
print(modelo_simples_cafe.summary())

# Gráfico de dispersão (sem reta automática: fit_reg=False)
sns.lmplot(
    data=dados,
    x='Preco_Cafe',
    y='Vendas_Cafe',
    hue='Promocao',
    palette={'Sim': 'red', 'Nao': 'black'},
    fit_reg=False,
    height=5,
    aspect=1.4
)
# Sobrepor a reta global do modelo simples
x_vals = np.linspace(dados['Preco_Cafe'].min(), dados['Preco_Cafe'].max(), 100)
y_vals = (modelo_simples_cafe.params['Intercept']
          + modelo_simples_cafe.params['Preco_Cafe'] * x_vals)
plt.plot(x_vals, y_vals, color='blue', linewidth=2,
         label='Linha de Regressão Simples')
plt.title('Linha de Regressão: Preço vs Vendas')
plt.legend()
plt.show()
```

Projeto Cafeteria: Regressão linear Múltipla

```
# C(Promocao) indica na fórmula que 'Promocao' é uma variável categórica

# Ajuste da regressão múltipla com 3 variáveis explicativas:
# Preco_Cafe (quantitativa), Promocao (categórica), Preco_Leite (quantitativa)
modelo_multiplo_cafe = smf.ols(
    'Vendas_Cafe ~ Preco_Cafe + C(Promocao) + Preco_Leite', data=dados).fit()

# Resumo da regressão múltipla
print(modelo_multiplo_cafe.summary())
```

Na fórmula do statsmodels, **C()** marca a variável como categórica e **cria as variáveis dummy automaticamente**
No resumo, o coeficiente aparece como **C(Promocao)[T.Sim]**: efeito da promoção em relação à categoria de referência ”Nao"

Uma **variável dummy** (ou **variável indicadora**) é uma variável numérica binária que representa categorias qualitativas com **valores 0 ou 1**.
É usada em modelos estatísticos, como **regressão linear múltipla**, para **incluir variáveis categóricas** (como sexo, cor, região) na análise, já que esses modelos requerem variáveis numéricas

Projeto Cafeteria: criar gráfico 3D

```
# Filtrar casos completos (sem valores ausentes)
dados_plot = dados[['Preco_Cafe', 'Preco_Leite', 'Vendas_Cafe']].dropna()

# Ajustar modelo com duas preditoras numéricas para o plano 3D
modelo_3d_cafe = smf.ols('Vendas_Cafe ~ Preco_Cafe + Preco_Leite', data=dados_plot).fit()

# Criar gráfico 3D com pontos
fig = plt.figure(figsize=(9, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(dados_plot['Preco_Cafe'], dados_plot['Preco_Leite'],
           dados_plot['Vendas_Cafe'], color='darkgreen', s=50)

# Malha para desenhar o plano de regressão
x_surf, y_surf = np.meshgrid(
    np.linspace(dados_plot['Preco_Cafe'].min(), dados_plot['Preco_Cafe'].max(), 20),
    np.linspace(dados_plot['Preco_Leite'].min(), dados_plot['Preco_Leite'].max(), 20)
)
z_surf = (
    modelo_3d_cafe.params['Intercept']
    + modelo_3d_cafe.params['Preco_Cafe'] * x_surf
    + modelo_3d_cafe.params['Preco_Leite'] * y_surf
)
# Adicionar plano de regressão
ax.plot_surface(x_surf, y_surf, z_surf, alpha=0.3, color='cyan')

ax.set_xlabel('Preço do Café')
ax.set_ylabel('Preço do Leite')
ax.set_zlabel('Vendas de Café')
ax.set_title('Plano de Regressão Múltipla (3D)')
plt.show()
```

Projeto Cafeteria

```
# Efeito marginal: previsão de vendas variando UMA variável e
# mantendo as demais fixas (preços na média, promoção em um nível)
def efeito(modelo, variavel, promocao='Nao', n=50):
    grade = pd.DataFrame({
        'Preco_Cafe': dados['Preco_Cafe'].mean(),
        'Preco_Leite': dados['Preco_Leite'].mean(),
        'Promocao': promocao
    }, index=range(n))
    grade[variavel] = np.linspace(dados[variavel].min(), dados[variavel].max(), n)
    previsao = modelo.get_prediction(grade).summary_frame(alpha=0.05)
    return grade[variavel], previsao

# Plota os efeitos lado a lado, com bandas de confiança de 95%
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, var in zip(axes, ['Preco_Cafe', 'Preco_Leite']):
    x, prev = efeito(modelo_multiplo_cafe, var)
    ax.plot(x, prev['mean'], color='blue')
    ax.fill_between(x, prev['mean_ci_lower'], prev['mean_ci_upper'], alpha=0.2)
    ax.set_xlabel(var)
    ax.set_ylabel('Vendas_Cafe')
fig.suptitle('Variáveis x Vendas Café')
plt.show()
```

O método **get_prediction()** do statsmodels calcula **a previsão média e o intervalo de confiança** variando uma variável e mantendo as demais fixas, o que facilita a **visualização e interpretação**

Projeto Cafeteria

```
# Visualizar Interações
# Se houver uma interação no modelo (ex: Preco_Cafe * C(Promocao)),
# você pode visualizar como o efeito do preço varia com a promoção ativa ou não:

# Ajustar modelo novamente, agora com o termo de interação
modelo_interacao = smf.ols(
    'Vendas_Cafe ~ Preco_Cafe * C(Promocao) + Preco_Leite', data=dados).fit()
print(modelo_interacao.summary())

# Plotar: uma reta (com banda de confiança) para cada nível de Promocao
sns.lmplot(
    data=dados,
    x='Preco_Cafe',
    y='Vendas_Cafe',
    hue='Promocao',
    palette={'Sim': 'red', 'Nao': 'black'},
    height=5,
    aspect=1.4
)
plt.title('Efeito de Interação: Preço do Café x Promoção')
plt.show()

# A inclinação da linha para 'Sim' vs 'Nao' mostra como a efetividade do preço
# muda com a promoção.
```

Projeto Cafeteria

```
# Analisando somente o efeito de uma variável
x, prev = efeito(modelo_interacao, 'Preco_Cafe', promocao='Nao')

plt.plot(x, prev['mean'], color='blue')
plt.fill_between(x, prev['mean_ci_lower'], prev['mean_ci_upper'], alpha=0.2)
plt.xlabel('Preco_Cafe')
plt.ylabel('Vendas_Cafe')
plt.title('Efeito do Preço do Café (Promocao = Nao)')
plt.show()

# "Para diferentes valores de Preco_Cafe, qual é a estimativa de Vendas_Cafe,
#  considerando que Promocao está em um nível fixo (ex: 'Nao') e
#  Preco_Leite está em seu valor médio?"
```

Projeto Cafeteria

```
modelo_completo = smf.ols(
    'Vendas_Cafe ~ Preco_Cafe + Preco_Leite + C(Promocao)', data=dados).fit()
modelo_sem_promocao = smf.ols('Vendas_Cafe ~ Preco_Cafe + Preco_Leite', data=dados).fit()

# Comparar efeitos marginais
fig, axes = plt.subplots(1, 2, figsize=(12, 4), sharey=True)
for ax, modelo, titulo in [(axes[0], modelo_completo, 'Com Promoção'),
                           (axes[1], modelo_sem_promocao, 'Sem Promoção')]:
    x, prev = efeito(modelo, 'Preco_Cafe', promocao='Sim')
    ax.plot(x, prev['mean'], color='blue')
    ax.fill_between(x, prev['mean_ci_lower'], prev['mean_ci_upper'], alpha=0.2)
    ax.set_title(titulo)
    ax.set_xlabel('Preco_Cafe')
axes[0].set_ylabel('Vendas_Cafe')
plt.show()

# Mesmo sob a condição de promoção ativa:
# O preço do café ainda impacta fortemente as vendas (quanto menor, mais vende);
# O preço do leite também afeta negativamente as vendas de café,
# possivelmente por serem consumidos juntos;
# A própria promoção tem um efeito positivo nas vendas, como esperado.
# Os gráficos representam efeitos marginais condicionais e mostram como cada variável
# influencia a resposta mantendo as outras fixas (Promocao = 'Sim').
```

Questões e análise do Projeto

- O preço do café influencia a venda?
- Promoções fazem diferenças?
- Quantas vezes as vendas ficaram acima/abaixo da média?
- Como é a relação entre a variável independente e as vendas? É crescente, decrescente ou curva?
- Há uma faixa da variável independente onde a previsão de vendas é menos confiável?
- Qual a variável tem maior efeito nas vendas?
- Analise o erro residual e o R2: os valores são bons ou ruins?Compare o resultado da regressão simples com a múltipla. Qual o melhor modelo?

O que você sugere para a cafeteria manter/ampliar as vendas de café?

Projeto Floricultura



- Você foi contratado por uma floricultura especializada em espécies do gênero *Iris*. A empresa deseja entender como diferentes características das flores se relacionam entre si, a fim de prever o **comprimento das pétalas** a partir de outras medições físicas.
- Para isso, você terá acesso a uma base de dados clássica da estatística chamada **iris**, que contém medições de 150 flores de três espécies (*setosa*, *versicolor* e *virginica*). Cada flor foi medida em relação a:
  - **sepal_length** – comprimento da sépala (em cm)
  - **sepal_width** – largura da sépala (em cm)
  - **petal_length** – comprimento da pétala (em cm)
  - **petal_width** – largura da pétala (em cm)
  - **species** – espécie da flor

Projeto floricultura

- Qual variável teve o maior impacto sobre o comprimento da pétala: o comprimento da sépala ou a largura da sépala?
- O modelo parece se ajustar bem visualmente aos dados?
- Como você interpretaria esse modelo para um colega da floricultura que não conhece estatística?

Projeto floricultura

```
# Carregar a base iris (nativa do seaborn)
iris = sns.load_dataset('iris')

# Visualização geral
print(iris.head())
print(iris.info())
print(iris.describe())

# Tabela de frequência para a variável categórica
print(iris['species'].value_counts())

# Gráficos exploratórios
plt.figure(figsize=(7, 4))
sns.boxplot(data=iris, x='species', y='petal_length', palette='pastel')
plt.title('Petal Length por Espécie')
plt.show()

sns.pairplot(iris, hue='species')   # matriz de dispersão
plt.show()
```

sns.pairplot()

- Mostra **visualmente a correlação** entre pares de variáveis.
- Ajuda a **identificar relações lineares, padrões e outliers**.
- Útil como **análise exploratória inicial** em conjuntos de dados multivariados.

Projeto floricultura

- O **matplotlib** cria **gráficos de dispersão em 3 dimensões** (3D) com projection='3d'
- útil para **visualizar relações entre três variáveis numéricas** em modelos exploratórios e até mesmo para ilustrar planos de regressão linear múltipla.

```
fig = plt.figure(figsize=(9, 6))
ax = fig.add_subplot(111, projection='3d')

# Atribuir cores por espécie
cores_iris = {'setosa': 'red', 'versicolor': 'green', 'virginica': 'blue'}

for especie, grupo in iris.groupby('species'):
    ax.scatter(grupo['sepal_length'],
               grupo['sepal_width'],
               grupo['petal_length'],
               label=especie,
               color=cores_iris[especie],
               s=45)

ax.set_xlabel('Sepal Length')
ax.set_ylabel('Sepal Width')
ax.set_zlabel('Petal Length')
ax.set_title('Gráfico 3D com matplotlib')
ax.legend()
plt.show()
```

Projeto Floricultura

```
# ------------------------
# Regressão Linear Simples
# ------------------------

mod_iris_simples = smf.ols('petal_length ~ sepal_length', data=iris).fit()
print(mod_iris_simples.summary())

# Gráfico com a reta de regressão
plt.figure(figsize=(7, 5))
sns.regplot(data=iris, x='sepal_length', y='petal_length',
            line_kws={'color': 'blue'})
plt.title('Regressão Simples: Petal Length vs Sepal Length')
plt.show()
```

Projeto Floricultura

```
# Ajustar modelo de regressão linear múltipla
mod_iris_multiplo = smf.ols('petal_length ~ sepal_length + sepal_width', data=iris).fit()
print(mod_iris_multiplo.summary())

# Criar gráfico 3D com pontos
fig = plt.figure(figsize=(9, 6))
ax = fig.add_subplot(111, projection='3d')
cores_iris = {'setosa': 'red', 'versicolor': 'green', 'virginica': 'blue'}
for especie, grupo in iris.groupby('species'):
    ax.scatter(grupo['sepal_length'], grupo['sepal_width'], grupo['petal_length'],
               label=especie, color=cores_iris[especie], s=45)

# Adicionar plano de regressão
x_surf, y_surf = np.meshgrid(
    np.linspace(iris['sepal_length'].min(), iris['sepal_length'].max(), 20),
    np.linspace(iris['sepal_width'].min(), iris['sepal_width'].max(), 20)
)
z_surf = (
    mod_iris_multiplo.params['Intercept']
    + mod_iris_multiplo.params['sepal_length'] * x_surf
    + mod_iris_multiplo.params['sepal_width'] * y_surf
)
ax.plot_surface(x_surf, y_surf, z_surf, alpha=0.25, color='gray')

# Extrair coeficientes e adicionar a equação no título do gráfico
eq_text = (f"Petal.Length = {mod_iris_multiplo.params['Intercept']:.2f} + "
           f"{mod_iris_multiplo.params['sepal_length']:.2f}*Sepal.Length + "
           f"{mod_iris_multiplo.params['sepal_width']:.2f}*Sepal.Width")
ax.set_title(f'Regressão Múltipla 3D\n{eq_text}', fontsize=10)
ax.set_xlabel('Sepal Length')
ax.set_ylabel('Sepal Width')
ax.set_zlabel('Petal Length')
ax.legend()
plt.show()
```

Projeto Floricultura

```
# Comparação com o modelo anterior
print(sm.stats.anova_lm(mod_iris_simples, mod_iris_multiplo))
```

Coluna	Significado
`df_resid`	Graus de liberdade residual de cada modelo
`ssr`	Soma dos quadrados dos resíduos (erro, RSS)
`df_diff`	Diferença nos graus de liberdade entre os modelos
`ss_diff`	Redução na soma dos quadrados (ganho explicativo)
`F`	Estatística F calculada para comparar os modelos
`Pr(>F)`	Valor-p: se for pequeno (< 0.05), o modelo mais complexo é significativamente melhor

Neste exemplo, observe o ssr (RSS, erro)

Projeto Floricultura

```
# -----------------------
# Inclusão de variável categórica e efeitos
# -----------------------

print(iris['species'].dtype)   # texto (object/str): entra na fórmula com C()

mod_iris_species = smf.ols('petal_length ~ sepal_length + C(species)', data=iris).fit()
print(mod_iris_species.summary())

# Visualização dos efeitos: ajuste separado por espécie
sns.lmplot(data=iris, x='sepal_length', y='petal_length',
           hue='species', height=5, aspect=1.4)
plt.title('Ajuste por Espécie (Controle do Paradoxo de Simpson)')
plt.show()
```

Atividade: concessionária

- Você trabalha em uma **concessionária de carros** e deseja entender como características técnicas dos veículos influenciam o **consumo de combustível (mpg - miles per gallon)**
- Você tem à disposição a base de dados mtcars, que contém informações de 32 modelos de carro, com variáveis como:
  - mpg: consumo (milhas por galão)
  - hp: potência do motor (horsepower)
  - wt: peso do carro
  - cyl: número de cilindros
  - am: tipo de transmissão (0 = automática, 1 = manual)

Atividade

- **Parte 0 – Estatística descritiva**
- Carregue a base com sm.datasets.get_rdataset('mtcars').data e use mtcars.describe() e mtcars.info() para explorá-la.
- Gere gráficos de dispersão com sns.scatterplot() ou sns.pairplot().
- **Parte 1 – Regressão Linear Simples**
- Ajuste um modelo de regressão linear simples prevendo mpg a partir de hp
- **Parte 2 – Regressão Linear Múltipla**
- Adicione o peso do carro como segunda variável

Atividade

- **Parte 3 – gráfico 3D**
  - Existe algum ponto extremo (outlier) que parece não se ajustar bem à reta/plano de regressão?

- **Parte 4 – Interpretação**
- Responda:
  - O que significa o sinal dos coeficientes?
  - O modelo com duas variáveis é melhor que o modelo simples? Justifique usando o sm.stats.anova_lm()
  - Que variáveis poderiam ser incluídas para melhorar o modelo?